

Computer Systems

COMP1071: Durham University

Matthew Emerson

April 2026

Contents

I	Digital Electronics	1
1	Circuits	1
1.1	Transistors	1
1.2	Circuit Design	1
1.3	Combinatorial Circuits	2
1.4	Sequential Circuits	2
2	Introduction to Logic	2
2.1	Functionally Complete Sets	2
II	Machine Architecture	3
3	MIPS	3
3.1	Introduction to MIPS	3
3.2	The MIPS Instruction Set	3
3.3	Memory Maps	3
4	The ATmega328P Microcontroller	4
4.1	Main Technical Characteristics	4
5	Programming in AVR Assembly	4
5.1	Directives	4
III	Databases	5
6	Database Design	5
6.1	The ANSI-SPARC Architecture	5
6.2	Designing a Database	5
7	Modelling Databases	6
7.1	Entity-Relationship Modelling	6
7.2	Relational Data Models	6
7.3	Functional Dependencies	6
7.4	Normalisation	7
8	Relational Calculus and Algebra	7
8.1	Database Languages	7
8.2	Relational Calculus	8
8.3	Relational Algebra	8
IV	Operating Systems	9
9	Introduction to Operating Systems	9
9.1	Basic Definitions	9
9.2	Operating System Design	9
9.3	Processes and Threads	10

10 Process, Memory, and Storage Management	10
10.1 Process Management	10
10.2 Memory Management	10
10.3 Storage Management	10
 V References	 11
11 Module Professors	11
12 Cited Works	11

Part I

Digital Electronics

1 Circuits

1.1 Transistors

Definition 1.1.1: Transistors are electronically controlled switches that turn on or off when voltage or current is applied to a control terminal.

Definition 1.1.2: A nMOS transistor is a sandwich of layers of conducting and insulating material. The gate is a conducting layer, on top of an insulating layer, on top of the substrate which is a semi-conductor. When its gate g is at a high voltage, current is allowed to flow from source to drain, and this is prevented when the gate is at a low voltage.

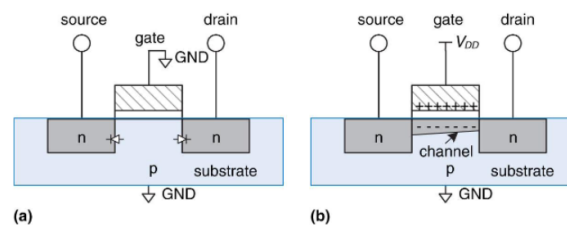


Figure 1: An nMOS transistor

The junction between p-type and n-type silicon is a diode. This means the current can only flow from p-type to n-type as they act as anodes and cathodes respectively.

Definition 1.1.3: A pMOS transistor is the opposite of a nMOS transistor - it allows current to flow when the gate is at low voltages, and prevents current flow at high voltages.

Definition 1.1.4: The static discipline states that if inputs to transistors meet valid thresholds, then the system must guarantee outputs will meet valid output thresholds and will not fall into the forbidden zone (outside of the accepted range for high/low voltages).

Definition 1.1.5: Moore's Law is usually quoted as "transistor density doubles every two years", or "computer processing power doubles every eighteen months".

1.2 Circuit Design

Definition 1.2.3: Abstraction is used when designing circuits to simplify problems by hiding unimportant details, highlighting the actual logic problem to be solved.

Definition 1.2.4: Another design principle used is discipline. Here, design choices are deliberately restricted to make circuits easier to model, design, and combine.

Definition 1.2.5: When designing complex systems, engineers use the three-y's: hierarchy, dividing a system into clear modules and submodules; modularity, using well-defined functions and interfaces for modules; and regularity, encouraging uniformity so modules can be swapped or reused.

1.3 Combinatorial Circuits

Definition 1.3.1: A combinatorial circuit is a circuit where the output depends purely on the current input.

Definition 1.3.2: An adder circuit takes the contents of two registers and adds them together.

- **Definition 1.3.3:** A half-adder circuit takes two one-bit binary inputs and adds them, returning a sum bit, and a carry bit.
- **Definition 1.3.4:** A full-adder circuit takes three one-bit binary inputs (one of which is a carry bit) and adds them, returning a sum bit, and a carry bit.

Definition 1.3.5: A decoder circuit takes an n -bit binary number and decodes it into 2^n data lines.

Definition 1.3.6: A multiplexor (mux) circuit uses a binary number to select one of a set of inputs to funnel to its output.

Definition 1.3.7: A tristate takes in two bits, an input bit and an enable bit. If the enable bit is a one, the input bit is outputted, if not then the output is left floating.

1.4 Sequential Circuits

Definition 1.4.1: A sequential circuit is a circuit which changes its output depending on its input history, alongside the current input.

Definition 1.4.2: A circuit which has two stable output states for a given input is known as a bistable circuit.

Definition 1.4.3: An sr-latch takes in two inputs, set S and reset R . If set has a pulse of one, then the output remains at one, even after the pulse ends. If reset has a pulse of one, then the output is reset back to zero. This is a bistable circuit.

Definition 1.4.4: A d-latch has two inputs, a data input D and a clock input CLK . The latch only changes to match the input at D when the CLK signal is high.

Definition 1.4.5: A d-type flip-flop works similarly to the d-latch, but only allows the stored output to change on the rising edge of the clock signal.

2 Introduction to Logic

2.1 Functionally Complete Sets

Definition 2.0.1: A functionally complete set is a set of boolean operators which can be used to express all possible truth tables by combining members of the set into a boolean expression. For example, $\{AND, OR, NOT\}$, $\{NOR\}$, and $\{NAND\}$ are all functionally complete sets.

Part II

Machine Architecture

3 MIPS

3.1 Introduction to MIPS

Definition 3.1.1: A MIPS (Microprocessor without Interlocked Pipeline Stages) is 32-bit RISC processor used in embedded systems and consumer electronics. It uses 32-bit words, has around 110 instructions, and 32 general purpose registers.

Definition 3.1.2: Amdahl's Law states that the performance improvement to be gained from using some faster mode of execution is limited by a fraction of the time the faster mode can be used.

3.2 The MIPS Instruction Set

The instruction set uses five addressing modes, these are:

- **Definition 3.2.1:** When all addresses in the instruction are registers, this is register-only addressing.
- **Definition 3.2.2:** When a value is used directly in an instruction, this is immediate addressing.
- **Definition 3.2.3:** Base addressing is when the address of a memory operand is given by a base address and a signed immediate.
- **Definition 3.2.4:** PC-relative addressing takes the current position in memory and jumps a given amount to a new address.
- **Definition 3.2.5:** Pseudo-direct addressing is used by MIPS in place of direct addressing as it does not support 32-bit addresses. As a result, it calculates it from 26 as the two least significant bits are always zeros as all words are multiples of four, and the four most significant bits can also be left as zero.

The MIPS instruction set can be split into three instruction types, these are:

- **Definition 3.2.6:** R-type instructions contain a six-bit opcode, three five-bit registers (two source and one destination), a five-bit shift value, and a six-bit code to identify the function.
- **Definition 3.2.7:** I-type instructions contain a six-bit opcode, two five-bit registers (one source, one destination), and a sixteen-bit immediate.
- **Definition 3.2.8:** J-type instructions contain a six-bit opcode and a twenty-six bit memory address. The compiler combines the two to form a new thirty-two bit address.

3.3 Memory Maps

MIPS uses 32-bit memory addresses across 2^{32} bits (or 4 GB), ranging from 0x0 to 0xFFFFFFFFC. This address space is split into the four segments:

- **Definition 3.3.1:** The text segment stores the machine language program and can store 256 MB.
- **Definition 3.3.2:** The global data segment stores global variables for program functions to access. These can be accessed using the pointer `$gp`. This segment can store 64 KB of data.
- **Definition 3.3.3:** The dynamic data segment stores data which is dynamically allocated/deallocated through the execution of the program and can store up to 2 GB of data using a stack and a heap.
- **Definition 3.3.4:** The reserved segment is used by the operating system and cannot be directly accessed by the program.

4 The ATmega328P Microcontroller

4.1 Main Technical Characteristics

Definition 4.1.1: A microcontroller is an integrated device consisting of a microprocessor, memory, and an input/output. They are used in embedded systems and store their program in flash memory.

The microcontroller has 32 general purpose 8-bit registers that are copied onto SRAM memory. Registers 26-31 are used in addressing, and as such should be treated as three 16-bit registers, rather than six 8-bit registers.

Definition 4.1.2: The status register is an 8-bit register separate from the other 32. Its eight bits are used as flags.

The microcontroller's pins are separated into 3 groups, B (digital), C (analogue), and D (digital).

Definition 4.1.3: The direction of each pin of a port is controlled by a data direction registers. We say a bit is set when it is 1 and the corresponding pin is output, and that a bit is cleared when it is 0 and the pin is input.

5 Programming in AVR Assembly

5.1 Directives

Definition 5.1.1: The assembly code contains statements called directives. These do not correspond to assembly instructions, but the assembler will take them into account when compiling. In AVR assembly, a directive is preceded by a dot. Some examples include:

- *.equ* assigns a value to a label which can be used in later expressions. This is a constant. (*.equ pi, 3*).
- *.global start* and *.global btnLED* declare the functions *start* and *btnLED* as global.

Part III

Databases

6 Database Design

6.1 The ANSI-SPARC Architecture

The architecture, set out by ANSI (the American National Standards Institute) and SPARC(the Standards, Planning, and Requirements Committee), consists of three levels used to realise data abstraction allowing different users to access the same data in different views without affecting each other. These levels are:

- **Definition 6.1.1:** The external layer is the user's view of the database. It can be adapted by the user to fit their specific needs without affecting others. This layer also incorporates derived, calculated, or combined fields.
- **Definition 6.1.2:** The conceptual layer is the community view of the database. It describes what data is stored in the database alongside the logical structure of the entire database.
- **Definition 6.1.3:** The internal layer is the physical representation of the database which describes how the data is stored to achieve optimal performance and storage utilisation.

Definition 6.1.4: A database schema is a complete description of the database as a result of the database design process. Each database should contain multiple external schemata, a conceptual schema, and an internal schema.

Definition 6.1.5: A database instance is the concrete data contained within the database at any particular time.

Definition 6.1.6: A database is considered to have logical data independence if the external schemata remain the same if the logical structure is changed on a conceptual level.

Definition 6.1.7: A database is considered to have physical data independence if the conceptual schema stays the same when the internal schema is changed.

6.2 Designing a Database

Definition 6.2.1: When creating a database, the first thing to do is to create a conceptual design. This is a high-level model of the data using entity-relationship modelling, identifying the appropriate entities, attributes, relationships, and constraints. This should be independent of any physical constraints.

Definition 6.2.2: The next step is to create a logical design which will be used to construct the relational data model using the conceptual design as a guide to transform entities and relationships into tables, using normalisation techniques to eliminate redundancies.

Definition 6.2.3: The physical design should be created last, and will describe how the logical design will be implemented, defining storage, access, and security details.

7 Modelling Databases

7.1 Entity-Relationship Modelling

Definition 7.1.1: An entity should be explicitly represented, denoted by a labelled rectangle and split into types and instances.

Definition 7.1.2: A relationship is a named association between two (or more) entity types which is represented by a labelled line with a triangle indicating reading direction.

Definition 7.1.3: Constraints are commonly represented on an entity-relationship model by a range ($min...max$). This can also include the cardinality (one-to-one, many-to-one, etc.), and participation (optional or mandatory).

Definition 7.1.4: The properties of an entity type are known as its attributes. In an entity-relationship model they are placed below an entities title with the primary key underlined.

7.2 Relational Data Models

Definition 7.2.1: A relational data model is based on the concept of mathematical relations and is the most widely used data structure for designing databases. It models a database by storing the relations between data in tables.

When creating a relational data model, the design must meet the following integrity constraints:

- **Definition 7.2.2:** Domain constraints for attributes limit the range of values permitted.
- **Definition 7.2.3:** Entity integrity ensures that the primary key is not null, guaranteeing that each entity has a unique identifier and that foreign keys form a valid reference.
- **Definition 7.2.4:** Referential integrity ensures that all foreign keys match some primary key or is null. This ensures that all references are valid.

7.3 Functional Dependencies

Definition 7.3.1: A relation's functional dependencies describe the relationship among its attributes.

Definition 7.3.2: If A and B are two sets of attributes in the same relation, where A is associated with exactly one value of B , then A functionally determines B and B is functionally dependent on A . A is called the determinant of B . Functional dependencies can take one of three forms:

- **Definition 7.3.3:** A full functional dependency exists when B is functionally dependent on A , but not on any proper subset of A .
- **Definition 7.3.4:** A partial functional dependency exists when B is functionally dependent on at least one proper subset of A .
- **Definition 7.3.5:** A transitive functional dependency exists between A and C if $A \rightarrow B$ and $B \rightarrow C$.

Definition 7.3.6: Given a set of functional dependencies F , its closure F^+ are all the functional dependencies which are implied by the dependencies in F . These can be calculated using Armstrong's Axioms.

Definition 7.3.7: Armstrong's Axioms are:

- **Definition 7.3.8:** Reflexivity: If $B \subseteq A$, then $A \rightarrow B$.
- **Definition 7.3.9:** Augmentation: If $A \rightarrow B$, then $(A, C) \rightarrow (B, C)$.
- **Definition 7.3.10:** Transitivity: If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$.

These rules are complete and sound, and can be used to derive another three rules:

- **Definition 7.3.11:** Decomposition: If $A \rightarrow (B, C)$, then $A \rightarrow B$ and $A \rightarrow C$.
- **Definition 7.3.12:** Union: If $A \rightarrow B$ and $A \rightarrow C$, then $A \rightarrow (B, C)$.
- **Definition 7.3.13:** Composition: If $A \rightarrow B$ and $C \rightarrow D$, then $(A, C) \rightarrow (B, D)$.

The closure F^+ of F can be computed by taking of F 's dependencies, repeatedly applying Armstrong's Axioms, and adding any new dependencies discovered in this way.

7.4 Normalisation

Definition 7.4.1: A table is in first normal form if it has no repeating groups (every cell has one value).

Definition 7.4.2: A table is in second normal form if it is in first normal form and has no partial dependencies (every non-key attribute is dependent on the whole primary key).

Definition 7.4.3: A table is in third normal form if it is in second normal form and has no transitive dependencies.

8 Relational Calculus and Algebra

8.1 Database Languages

Definition 8.1.1: Structured query language is the most common database language. It follows the ISO/ANSI standard using simple syntax with its two components, DDL and DML:

- **Definition 8.1.2:** Data definition language is used to define the schema for each relation, the domain for each respective attribute, and to specify the relation's integrity constraints.
- **Definition 8.1.3:** Data manipulation language is used to insert, update, delete, and retrieve data from the database.

Database languages can be separated into one of two types:

- **Definition 8.1.4:** A declarative language specifies which data needs to be retrieved in a query (relational calculus).
- **Definition 8.1.5:** A procedural language specifies how to retrieve the requested data for the query (relational algebra).

Part IV

Operating Systems

9 Introduction to Operating Systems

9.1 Basic Definitions

Definition 9.1.1: An operating system is a program that acts as an intermediary between a user and the hardware. Its goal is to execute programs, make solving problems easier, make the computer system convenient to use, and to use the resources of the system fairly and efficiently. The operating system is:

- **Definition 9.1.2:** A resource allocator. It is responsible for the management of the computer system's resources.
- **Definition 9.1.3:** A control program. It controls the execution of user programs and operation of the input/output devices to prevent errors and improper use of the computer.
- **Definition 9.1.4:** A kernel. It is the one program that runs all of the time.

9.2 Operating System Design

Definition 9.2.1: The operating system generally runs in dual-mode operation. This allows the operating system to protect itself and other system components. It differentiates which mode it is currently in using the mode bit.

- **Definition 9.2.2:** When the computer is in user mode it is limited as to which instructions it can execute, with some requiring switching modes to,
- **Definition 9.2.3:** Kernel mode (also known as system or privileged mode). In this mode, the system is given access to a set of privileged instructions which can perform tasks such as managing interrupts, performing inputs/outputs, and halting a process.

The operating system offers a set of services, some of which are:

- Providing an environment for the execution of programs and services.
- Providing a user interface, which can either be on the command line (CLI), graphical (GUI), or in Batch.
- Controlling the operation of input/output devices.
- Manipulating the file system.
- Organising communications either between processes on the same computer, or between computers over a network.
- Providing error detection, taking appropriate action to ensure correct and consistent computing.
- Allocating resources, particularly when multiple jobs are running concurrently.
- Providing protection and security by ensuring that all access to system resources are controlled and that all users are authenticated.

9.3 Processes and Threads

10 Process, Memory, and Storage Management

10.1 Process Management

10.2 Memory Management

10.3 Storage Management

Part V

References

11 Module Professors

The below is accurate as of the 2025/26 academic year and will not be updated in subsequent years.

- *Digital Electronics* and *Machine Architecture* are taught by Ioannis Ivrissimtzis.
 - **Office Location:** Room 2023, MCS Building
 - **Preferred Contact Methods:** ioannis.ivrissimtzis@durham.ac.uk
 - **Pronouns:** he, him
- *Databases* is taught by Peter Davies-Peck.
 - **Office Location:** Room 2063, MCS Building
 - **Preferred Contact Methods:** peter.w.davies@durham.ac.uk
 - **Pronouns:** he, him
- *Operating Systems* is taught by Anish Jindal.
 - **Office Location:** Room 2065, MCS Building
 - **Preferred Contact Methods:** anish.jindal@durham.ac.uk
 - **Pronouns:** he, him

12 Cited Works